

SAM — VIVA VOCE & PROJECT DEFENSE

Q&A GUIDE

Comprehensive Examination & Defense Reference

Target: SBTET / Polytechnic Institutions

Department: Computer Science and Engineering

Stack: React 19 • TypeScript • Vite 8 • Firebase • Cloudinary

Status: 184/184 Automated Tests Passed

SAM — VIVA VOCE & PROJECT DEFENSE Q&A GUIDE

Project Title: SAM (Smart Assignment Manager)

Target Audience: Polytechnic Diploma / Engineering Project Examination Committee & Review Panels

Verification Status: 100% Code-Verified (184/184 Automated Tests Passed)

SECTION 1: PROJECT CONCEPT & ARCHITECTURE

Q1: What is SAM and why did you build it?

Answer:

"SAM stands for **Smart Assignment Manager**. It is a Progressive Web Application (PWA) developed to solve the logistical and integrity challenges of handwritten assignments in polytechnic diploma institutions.

In our diploma curriculum, handwritten assignments are mandatory. However, physical notebooks frequently get lost or damaged, carrying stacks of notebooks is exhausting for staff, and traditional digital platforms allow students to upload recycled photos or share screenshots.

SAM provides an authenticated, paperless, and tamper-resistant workflow: students photograph their physical handwritten notebook pages with their phone camera, SAM burns a dynamic verification code and watermark badge into the image, compresses it to ~150 KB, and uploads it to Cloudinary. Teachers then evaluate the submitted pages inside a full-screen digital workspace with a digital red pen, enter marks out of 10, and provide structured feedback."

Q2: Why did you choose React 19 and Vite 8 instead of traditional technologies?

Answer:

"1. **Component-Based Architecture:** The interactive canvas overlay, live camera preview, and dynamic assignment cards require reactive, reusable UI components.

1. **Speed & Efficiency with Vite:** Vite leverages native ES modules and Rollup/Rolldown, compiling over 2000 modules in under 500 milliseconds, ensuring instant updates during development and lightweight production bundles.
 2. **Type Safety with TypeScript:** Prevents runtime errors by strictly enforcing domain types across assignments, submissions, annotations, and user roles."
-

Q3: Why did you choose Firebase Authentication and Cloud Firestore?

Answer:

"1. **Authoritative Identity (UID):** Firebase Auth manages encrypted user sessions and provides an immutable `user.uid` that acts as the authoritative anchor for all security rules and database queries.

1. **Real-time NoSQL Flexibility:** Cloud Firestore offers document-based storage ideal for hierarchical academic entities (such as multi-round submission histories and vector annotations).
 2. **Declarative Security Rules:** With Firestore Security Rules, authorization is enforced at the database level—even if a client requests an unauthorized document, Firestore rejects the request."
-

Q4: Why did you choose Cloudinary instead of storing images directly in Firestore or Firebase Storage?

Answer:

"1. **Firestore Document Size Limit:** Firestore documents have a strict 1 MB size limit. Storing raw image Base64 strings directly in Firestore would quickly exceed document limits and degrade database read performance.

1. **Direct Unsigned Client Uploads:** Cloudinary supports secure unsigned upload presets, allowing student mobile browsers to upload directly to the cloud without needing a dedicated backend server or exposing private API secrets.
 2. **Global CDN & Responsive Delivery:** Cloudinary automatically provides fast, encrypted HTTPS Content Delivery Network (CDN) URLs (`secure_url`), ensuring high-resolution pages load smoothly in the teacher's evaluation workspace."
-

SECTION 2: AUTHENTICATION & SECURITY

Q5: How do students and teachers log in without standard institutional email addresses?

Answer:

"We implemented a **Synthetic Email Synthesis Engine** in `authService.ts` :

- Students log in using their official Diploma Registration PIN (e.g. `24170-CM-001`) and password. SAM translates this into `24170-cm-001@student.sam.edu.in`.
- Staff and Admin log in using their 10-digit Indian mobile number and password. SAM translates this into `staff_@sam.edu.in` OR `admin_@sam.edu.in`.

This allows us to leverage Firebase Authentication's industry-standard cryptographic sessions without forcing the institution to create and maintain separate Google or email accounts for thousands of students."

Q6: How does SAM prevent Student A from viewing or tampering with Student B's submissions?

Answer:

"Security is enforced at two distinct levels:

1. **Frontend Isolation:** All queries for student assignments and submissions strictly filter by `auth.currentUser.uid`.
2. **Database Security Rules:** In `firestore.rules`, the `submissions` collection permits read and create operations only when:

```
```javascript
```

```
request.auth.uid == resource.data.studentId
```

```
```
```

Furthermore, students are strictly forbidden from writing or altering the `marks` field:

```
```javascript
```

```
request.resource.data.marks == resource.data.marks
```

```
```
```

This makes it cryptographically impossible for Student A to read or alter Student B's data."

Q7: What happens when multiple students use the same desktop computer in a college lab?

Answer:

"SAM implements strict **Same-Device Isolation** in `App.tsx` and `AuthContext.tsx`. When a student clicks Logout:

1. Firebase calls `signOut(auth)`.
 2. In-memory state, active drilldowns (`activeAssignmentId`, `activeClassId`), and session caches are completely purged.
 3. When the next student logs in, Firebase's `onAuthStateChanged` fires with the new UID, and data fetching queries run strictly for the new user. No cached data from the previous student is ever retained."
-

SECTION 3: SUBMISSION, CAMERA & WATERMARKING

Q8: Why is the new submission workflow camera-only? Why did you remove file uploads and gallery pickers?

Answer:

"To prevent academic dishonesty. When file pickers or gallery uploads are permitted, students can easily:

- Share photo files via WhatsApp or Telegram.
- Upload screenshots of someone else's work.
- Submit recycled photos from previous years.

By enforcing `navigator.mediaDevices.getUserMedia` with the rear camera (`facingMode: { ideal: "environment" }`), the student must physically point the camera at their own handwritten notebook page. Watermarks and verification codes are burned into the image pixels in real-time."

Q9: How does the Verification Code work?

Answer:

"When a student opens an assignment, the system generates a unique **6-character alphanumeric code** (e.g., `K74821`):

1. **Charset:** Uses 32 unambiguous characters (`23456789ABCDEFGHIJKLMNPQRSTUVWXYZ`), excluding visually confusing characters (`0` , `o` , `1` , `I`).
 2. **Cryptographic Randomness:** Generated using the browser's `crypto.getRandomValues()` API.
 3. **Persistence:** Saved in `/verification_codes/vcode_{assignmentId}_{studentId}` . The code is permanent for that specific student and assignment, remaining identical across app reloads, retakes, and resubmissions.
 4. **Physical Requirement:** The student writes this code in ink at the top of their notebook page before taking the photo."
-

Q10: What information is in the watermark, and how is it applied?

Answer:

"The watermark is rendered client-side on an HTML5 canvas before the image is compressed or uploaded. It draws:

- Student Name (e.g. `S. Rajesh Kumar`)
- Diploma PIN (e.g. `PIN: 24170-CM-001`)
- Verification Code (e.g. `Code: K74821`)
- Timestamp (e.g. `24 Sep 2026, 11:15 am`)

It is placed in a high-contrast, translucent dark pill badge in the safe bottom-right corner of the canvas so it never obscures the student's handwritten answers."

Q11: Why do you compress images, and what are the compression specifications?

Answer:

"High-resolution mobile photos are typically 4 MB to 8 MB each. Uploading multiple uncompressed pages would crash mobile data limits and slow down the teacher's grading workspace.

In `imageCompressor.ts`:

- **Max Dimension:** Scaled down to a maximum of 1600 pixels.
- **Progressive JPEG Quality:** Bounded between 0.55 and 0.82.
- **Target File Size:** Approximately 150 KB per page.

This 95% size reduction preserves sharp, dark ink strokes and page margins while enabling instant mobile uploads."

SECTION 4: TEACHER EVALUATION & DIGITAL ANNOTATION

Q12: Explain the Teacher Evaluation Screen. What makes it special?

Answer:

"We built a **Full-Screen 100vw × 100vh Digital Workspace** (`GradeSubmissionModal.tsx`) that replaces standard small popups:

1. **Dedicated Workspace:** Provides maximum screen area to inspect handwritten student work.
 2. **Fixed Header:** Contains student metadata, PIN, verification code, and a persistent `DONE` button.
 3. **Transparent Canvas Overlay** (`AnnotationCanvas.tsx`): A drawing layer positioned on top of the student's page. The teacher can draw red pen checkmarks, cross out mistakes, and write feedback.
 4. **Queue Navigation:** The teacher can navigate from student to student without returning to the dashboard.
 5. **Non-Destructive:** The student's original Cloudinary image is never overwritten. Annotation strokes are stored as separate vector points in `/submissionAnnotations/{submissionId}`."
-

Q13: What annotation tools are available to the teacher?

Answer:

"1. **Pen Tool:** 4 colors (Red `#EF4444`, Blue `#3B82F6`, Green `#10B981`, Black `#111827`) and 3 stroke widths (Thin 2px, Medium 4px, Thick 7px). Default is Red Medium.

1. **Vector Eraser:** Cleans up teacher markings without touching the underlying notebook photo.
 2. **Text Annotation Tool:** Enables clicking anywhere on the page to type notes, with one-click presets like 'Check calculation' or 'Good diagram'.
 3. **Multi-Level Undo/Redo:** Full history tracking with keyboard shortcuts (`Ctrl+Z`, `Ctrl+Y`).
 4. **Synchronized Zoom & Pan:** 0.5x to 3.0x zoom; image and drawings scale together in exact alignment."
-

Q14: How does the Return and Resubmission cycle work?

Answer:

"If a teacher finds mistakes in an assignment:

1. The teacher selects **Return for Correction** and provides mandatory feedback.
 2. The submission status updates to `'returned'`.
 3. The student receives a notification and sees a red `RETURNED` badge on their dashboard.
 4. The student clicks **Resubmit Assignment**. The system retains the original verification code.
 5. When the student uploads new pages, the system archives the previous round's images, marks, feedback, and timestamps into the `history: SubmissionHistoryItem[]` array.
 6. The teacher can review both the new work and the complete audit trail of prior attempts."
-

SECTION 5: DATABASE & LIFECYCLE MANAGEMENT

Q15: What is the difference between Assignment Status and Student Submission Status?

Answer:

"They govern two different lifecycles:

1. Assignment Lifecycle (`status` on `/assignments`):

- `draft` : Created by teacher, hidden from students.
- `published` : Active, visible to targeted class students.
- `closed` : Past due date or closed by teacher; blocks new submissions.
- `archived` : Historical record.

1. Student Submission Lifecycle (`status` on `/submissions`):

- `not_submitted` : Student has not yet captured pages.
 - `submitted` : Student uploaded pages; awaiting teacher review.
 - `under_review` : Teacher opened the paper in the evaluation workspace.
 - `checked` : Teacher evaluated and entered final marks.
 - `returned` : Teacher sent the paper back for student correction."
-

Q16: How does assignment class targeting work?

Answer:

"Assignments are targeted by academic attributes: `semester` (`1st` , `3rd` , `4th` , `5th`), `department` (`CSE`), and `classId` .

When a student logs in, their profile contains their enrolled semester. The dashboard queries only assignments where `assignment.semester == student.semester` . Even if a student bypasses the UI, `submissionService.submitAssignment()` checks:

TYPESCRIPT

```
if (student.semester !== assignment.semester) {  
  return { error: 'Unauthorized semester mismatch' };  
}
```

This guarantees complete class isolation."

SECTION 6: TESTING & DEMONSTRATION SUMMARY

Q17: What automated testing did you implement, and what were the results?

Answer:

"We developed 10 automated test suites executed via Node.js:

1. `test_branding.mjs` : Branding and institution tokens (10 tests)
2. `test_scenarios.mjs` : Multi-user scenarios and class isolation (15 tests)
3. `test_camera_flow.mjs` : Camera API, watermarking, and compression (29 tests)
4. `test_verification_code.mjs` : Cryptographic code generation and persistence (17 tests)
5. `test_admin_system.mjs` : Admin provisioning and academic allocations (20 tests)
6. `test_assignment_system.mjs` : Assignment drafting, publishing, and targeting (25 tests)
7. `test_submission_grading.mjs` : Submission, grading, and history preservation (33 tests)
8. `test_new_submission_no_drive.mjs` : Camera-only enforcement (14 tests)
9. `test_ai_verification.mjs` : OCR handwriting code matching (6 tests)
10. `test_fullscreen_evaluation.mjs` : Full-screen annotation workspace (15 tests)

Total Results: Exactly 184 / 184 tests passed with zero failures."

Q18: What are the current limitations of SAM?

Answer:

"1. **Camera Dependency:** Since the new submission system is camera-only, students must have a working device camera.

1. **Network Dependency for Final Sync:** Image capture, canvas watermarking, and compression happen offline in browser memory, but uploading to Cloudinary and Firestore requires internet connectivity.
 2. **Curricular Focus:** Pre-configured out of the box for Computer Science and Engineering (`CSE`). Other departments require the Admin to provision their respective subjects."
-

Q19: What are your planned future enhancements?

Answer:

"1. **Automated AI Grading Assistant:** Using vision-language models to provide teachers with an initial handwriting transcription and suggest preliminary marks.

1. **Bulk PDF Archival Export:** Allowing institutional administrators to export an entire class's evaluated assignments as a single audit-ready PDF package for accreditation.
2. **WebPush Notifications:** Native background alerts on mobile devices when assignments are published or returned."